

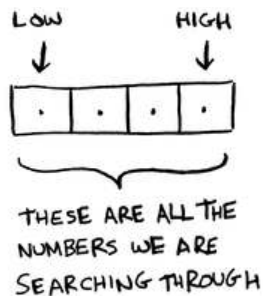
Note

Binary search only works when your list is in sorted order. For example, the names in a phone book are sorted in alphabetical order, so you can use binary search to look for a name. What would happen if the names weren't sorted?

Let's see how to write binary search in Python. The code sample here uses arrays. If you don't know how arrays work, don't worry; they're covered in the next chapter. You just need to know that you can store a sequence of elements in a row of consecutive buckets called an array. The buckets are numbered starting with 0: the first bucket is at position #0, the second is #1, the third is #2, and so on.

The `binary_search` function takes a sorted array and an item. If the item is in the array, the function returns its position. You'll keep track of what part of the array you have to search through. At the beginning, this is the entire array:

```
low = 0
high = len(list) - 1
```



Each time, you check the middle element:

```
mid = (low + high) / 2
guess = list[mid]
```

mid is rounded down by Python automatically if (low + high) isn't an even number.

If the guess is too low, you update `low` accordingly:

```
if guess < item:
    low = mid + 1
```